

Отчет по проекту «Оркестрация моделей в Triton Inference Server»

Описание

Данный проект реализован и протестирован полностью локально на следующем железе:

Intel Core Ultra 9, 32 gb CPU, Nvidia GeForce RTX 5070 Laptop 8gb GPU, Windows 11 Pro
(использовалась среда WSL с установленной Ubuntu 24.04)

Такая характеристика железа привела к некоторым изменениям при работе с проектом, в частности, классификатор запускался на CPU.

Внимание! На Github в каталоге `\model_repository\classifier\1` лежат файлы `model.z01` и `model.zip`, это заархивированный и разбитый на две части onnx файл `model.onnx`, он целиком не помещался в загрузки на Github. Если планируется запуск моего проекта, эти файлы на целевой системе надо разархивировать в один.

Проект представляет собой **гетерогенный ML-пайплайн**, развёрнутый в NVIDIA Triton Inference Server. Система принимает текстовый запрос от пользователя и возвращает осмысленный ответ, используя три независимые модели, оркеструемые через **Business Logic Scripting (BLS)**.

Логика работы:

1. Пользователь отправляет текст через HTTP-клиент.
2. orchestrator получает запрос и последовательно вызывает три модели:
 - `embedder` — получает векторное представление текста (для проверки пайплайна).
 - `classifier` — определяет, является ли текст вопросом.
 - `generator` — если это вопрос, генерирует ответ через LLM.
3. Результат возвращается клиенту.

Модели

Модель	Назначение	Бэкенд	Формат	Устройство
<code>classifier</code>	Детектор вопросов (0 — утверждение, 1 — вопрос)	<code>onnxruntime</code>	ONNX	CPU
<code>embedder</code>	Векторизация текста (384-dim)	<code>python</code>	SentenceTransformer (PyTorch)	GPU

Модель	Назначение	Бэкенд	Формат	Устройство
generator	Генерация ответа на вопрос	python	TinyLlama-1.1B (HuggingFace)	GPU
orchestrator	BLS-дирижёр всей цепочки	python	Кастомный код	GPU

Детали моделей

classifier — shahrukhx01/bert-mini-finetune-question-detection

- Небольшой BERT, обученный различать вопросы и утверждения.
- Экспортирован в ONNX через optimum-cli export onnx --task text-classification.
- Входы: input_ids, attention_mask, token_type_ids (форма [-1, 128], INT64).
- Выход: logits (форма [-1, 2], FP32).
- Запущен **на CPU** для стабильности в WSL2 (на GPU возникала ошибка cudaIpcHandle: invalid resource handle).

embedder — sentence-transformers/all-MiniLM-L6-v2

- Лёгкий SentenceTransformer, превращает текст в вектор размерности 384.
- Вход: text (BYTES, форма [1]).
- Выход: embedding (FP32, форма [1, 384]).
- Загружается один раз в initialize(), кэшируется в памяти.

generator — TinyLlama/TinyLlama-1.1B-Chat-v1.0

- Лёгкая LLM на 1.1B параметров, помещается в GPU-память.
- Вход: prompt (BYTES, форма [1]).
- Выход: text_output (BYTES, форма [1]).
- Параметры генерации: max_new_tokens=150.

orchestrator — кастомная BLS-модель

- Не содержит ML, только логику вызовов.
- Внутри execute() использует pb_utils.InferenceRequest(...).exec() для вызова других моделей **внутри Triton, без HTTP**.
- Поддерживает preferred_memory=TRITONSERVER_MEMORY_CPU — обход CUDA IPC для стабильности в WSL2.

orchestrator/model.py — ключевой файл проекта. Основные методы:

initialize(args) — вызывается один раз при загрузке:

- Загружает токенизатор shahrukh01/bert-mini-finetune-question-detection.
- Устанавливает `model_max_length = 128`.

`execute(requests)` — вызывается на каждый запрос:

1. Читает входной тензор `text` через `pb_utils.get_input_tensor_by_name`.
2. **Вызывает** `embedder` через `pb_utils.InferenceRequest(...).exec()`.
3. **Токенизирует текст** тем же токенизатором, что и у `classifier`.
4. **Вызывает** `classifier` через BLS с тремя тензорами: `input_ids`, `attention_mask`, `token_type_ids`.
5. Анализирует `logits` → `predicted_class` (0 или 1).
6. Если **класс = 1** (вопрос) — формирует промпт "Question: {text}\nAnswer:" и **вызывает** `generator`.
7. Возвращает финальный ответ как тензор `response` (BYTES).

Ключевая деталь: все вызовы используют `preferred_memory=TRITONSERVER_MEMORY_CPU` — это обход проблемы с CUDA IPC в WSL2.

- Используется **зеркало** `hubimage/nvidia-tritonserver:24.08-py3` — официальный `nvcf.io` недоступен из региона.
- `numpy<2` критичен: NumPy 2.x ломает сериализацию тензоров в BLS.

Порты:

- 8000 — HTTP (используется клиентом и `perf_analyzer`).
- 8001 — gRPC.
- 8002 — метрики Prometheus.

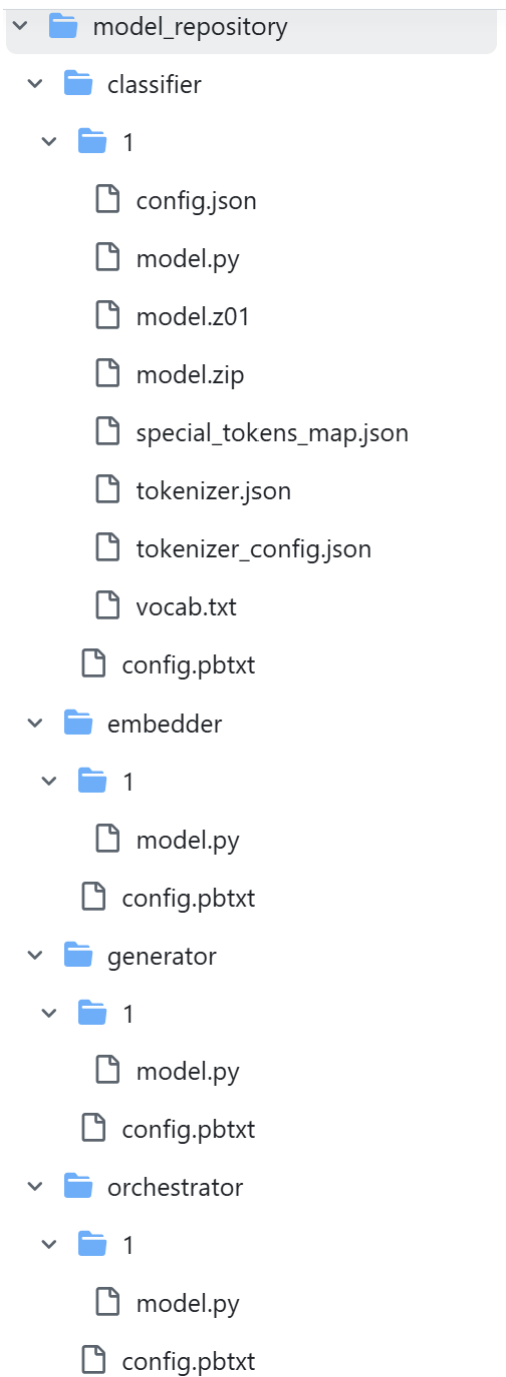
`client.py` — минимальный HTTP-клиент на `tritonclient[http]`

Использовался **Perf Analyzer** (входит в Triton SDK контейнер):

Для каждой модели замерялись:

- **Throughput** (infer/sec) — пропускная способность.
- **p95 Latency** — 95-й перцентиль времени ответа.
- **Server-side breakdown:** Queue / Compute Input / Compute Infer / Compute Output.

Иерархическое дерево Model Repository



Каждая модель версионирована папкой 1/, что соответствует стандарту Triton.

Отчёт о тестировании: Скриншоты или текстовые логи работы Perf Analyzer с итоговыми цифрами Latency и Throughput.

Текстовые логи работы Perf Analyzer находятся в корневом каталоге в папке **perf_data**

Name
 ..
 classifier_results.txt
 embedder_results.txt
 generator_results.txt
 metrics_during_test.txt
 orchestrator_results.txt
 prompt_data.json
 text_data.json

Classifier_results.txt – результаты классификатора

Embedder_results.txt – результаты эмбеддера

Generator_results.txt – результаты генератора

Orchestrator_results.txt – результаты всей цепочки

Графики или таблицы со сравнением производительности моделей.

Сводная таблица (concurrency=1)

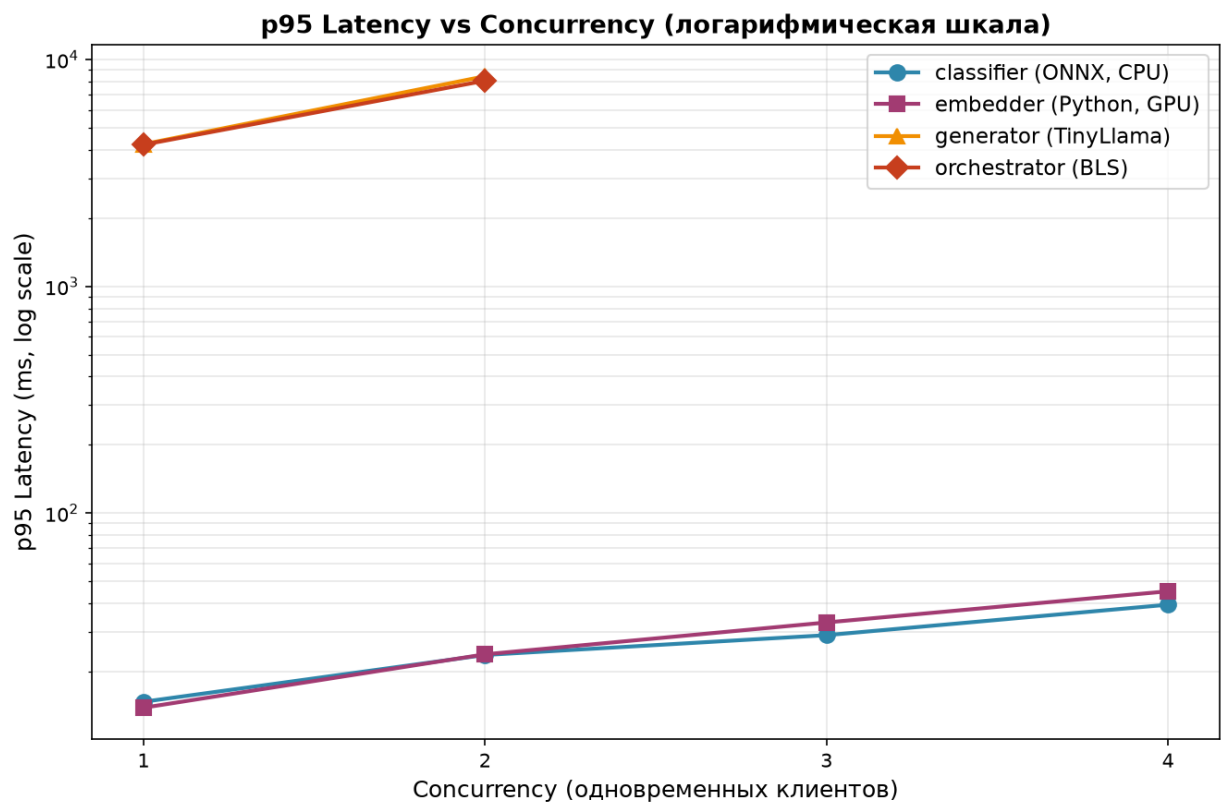
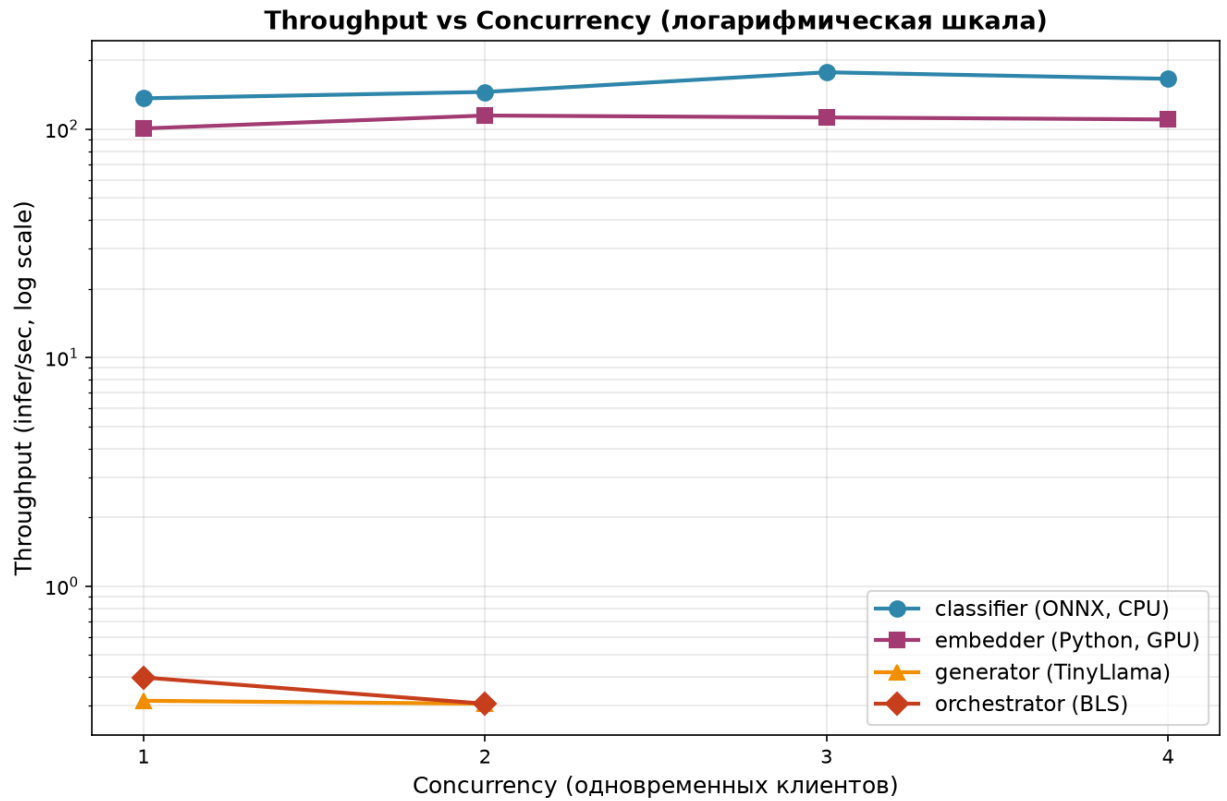
Модель	Throughput (RPS)	p95 Latency	Compute Infer	Queue	Узкое место
classifier	136–177	14.7 мс	5.4–6.7 мс	~0 мс	CPU

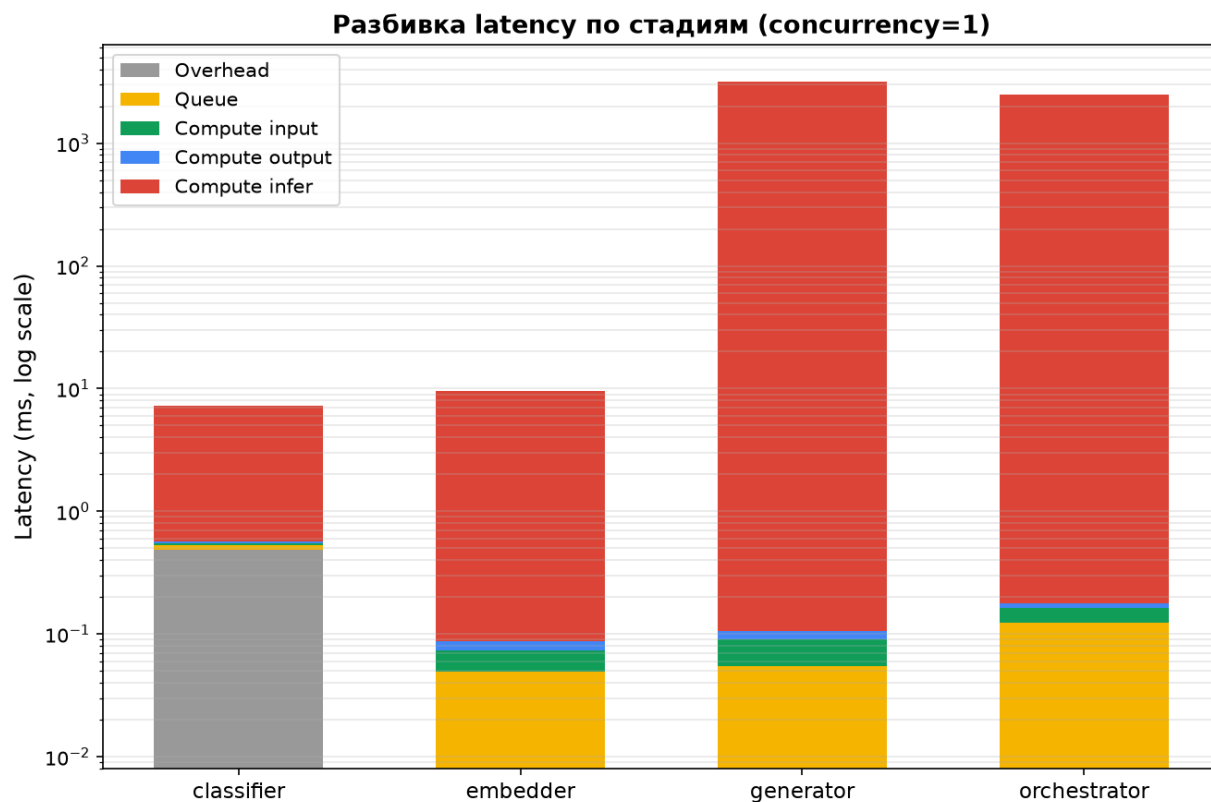
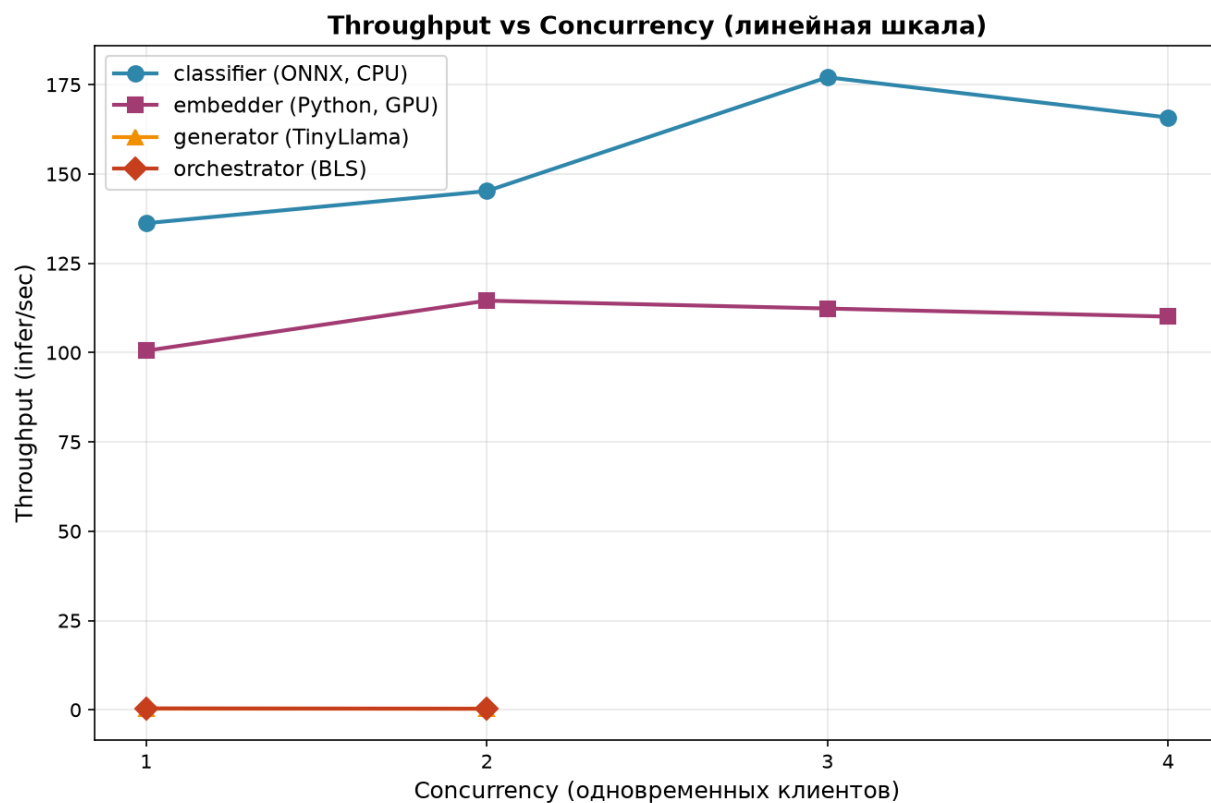
Модель	Throughput (RPS)	p95 Latency	Compute Infer	Queue	Узкое место
embedder	100–114	13.8 мс	8.6–9.4 мс	~0 мс	GPU compute
generator	0.31	4.2 сек	3.16 сек	~0 мс	LLM-генерация
orchestrator	0.40	4.2 сек	2.47 сек	~0 мс	генератор внутри

Соотношение скоростей:

- classifier быстрее generator примерно в **450 раз**
- embedder быстрее generator примерно в **330 раз**
- Вся BLS-цепочка ограничена **генератором**

Графики





Анализ по каждой модели

Classifier (ONNX, CPU, BERT-mini)

Что видно:

- RPS растёт: 136 → 145 → **177** → 166 (пик на concurrency=3)
- На concurrency=4 — **деградация** (166 RPS, latency 39 мс)
- compute infer стабилен ~5.5–6.7 мс — **не зависит от concurrency**, значит упирается в CPU
- queue растёт линейно: 0 → 6.5 → 10.6 → 17.5 мс

Вывод:

Модель хорошо масштабируется до concurrency=3, дальше упирается в CPU-ядра. Пиковая пропускная способность — **177 RPS**, что для BERT-mini на CPU в WSL2 — отличный результат.

Embedder (Python backend, GPU, SentenceTransformer)

Что видно:

- RPS: 100 → **114** → 112 → 110 (пик на concurrency=2)
- compute infer ~8.6–9.4 мс — стабилен
- queue растёт быстро: 0 → 8.3 → 17.4 → 26.9 мс

Вывод:

Embedder насыщается уже при concurrency=2, дальше throughput падает. Узкое место — **compute infer на GPU**. Пиковая пропускная способность — **114 RPS**.

Generator (Python backend, GPU, TinyLlama-1.1B)

Что видно:

- RPS: **0.31** при обоих concurrency — **не масштабируется**
- compute infer = **3.16 секунды** — главный тормоз
- На concurrency=2: latency 8.4 сек, queue 3.1 сек — запросы стоят в очереди

Вывод:

Это главное узкое место всей системы. TinyLlama-1.1B на GPU генерирует ~50–150 токенов за 3+ секунды, и параллельные запросы только увеличивают очередь. Throughput **не растёт** с concurrency — модель полностью утилизует GPU.

Orchestrator (BLS, вся цепочка)

Что видно:

- RPS: **0.40** — почти как у generator
- compute infer = **2.47 сек** (меньше, чем у generator)

Почему orchestrator быстрее generator?

Orchestrator шлёт генератору **короткие промпты** (text_data.json — просто «What is machine learning?»), а generator-тест шёл с длинными (Question: ... Answer:). Чем короче промпт → быстрее генерация.

Вывод:

Вся BLS-цепочка ограничена **генератором**. Embedder и classifier отработывают за ~10 мс, а generator — за 2.5–3 секунды. **Доля генератора в latency цепочки = 99.6%**. Именно его нужно оптимизировать.

Сделайте вывод: какие преимущества дает использование Triton и BLS по сравнению с обычным Flask/FastAPI сервисом? Предложите способы оптимизации (например, включение Dynamic Batching или Instance Groups), если задержки оказались слишком высокими.

Сравнение с Flask/FastAPI

Почему Triton лучше самописного сервиса:

Аспект	Flask/FastAPI	Triton + BLS
Управление моделями	Вручную	Model Repository, версионирование
Гетерогенные бэкенды	Нужно писать обёртки	Из коробки (ONNX, PyTorch, TensorRT, Python)
BLS-оркестрация	HTTP-запросы между сервисами (медленно)	Прямые вызовы через pb_utils.InferenceRequest (in-process)
Динамический батчинг	Реализовывать вручную	Флаг dynamic_batching в конфиге

Аспект	Flask/FastAPI	Triton + BLS
Метрики	Prometheus надо ставить отдельно	Порт 8002 из коробки
GPU sharing	Свой код	Instance Groups, scheduling
Масштабирование	uWSGI/gunicorn + несколько сервисов	Один сервер, N инстансов модели

Предложения по оптимизации

- Для **generator** (главный фикс):
 - Заменить на **vLLM** или **TensorRT-LLM** — это даст **10–50× ускорение** (по документации vLLM).
 - Уменьшить `max_new_tokens` (сейчас 150, можно до 100).
 - Использовать **квантизацию** (INT8/INT4) — уменьшит время инференса в 2–4 раза.
 - Взять модель поменьше (Qwen-0.5B, Phi-2).
- Для **classifier** и **embedder**:
 - Включить **dynamic batching** — увеличит throughput.
 - Увеличить `instance_group.count` — параллельные инстансы модели.
 - Для classifier: перевести обратно на GPU (когда решится проблема с WSL2).
- Для **orchestrator**:
 - Параллельные вызовы embedder и classifier (сейчас последовательно) — сэкономит ~10 мс.
 - Использовать `async_exes` для неблокирующих вызовов.

Пример отработавшего запроса со стороны Triton

```
I0915 13:13:44.723300 I model.py:125] "Generator done."
I0915 13:13:44.724492 I model.py:25] "Processing: What is a transformer model?..."
I0915 13:13:44.724679 I model.py:34] "Calling embedder..."
I0915 13:13:44.739598 I model.py:49] "Embedder done."
I0915 13:13:44.739743 I model.py:58] "Tokenizing for classifier..."
I0915 13:13:44.740216 I model.py:64] "Tokenization done."
I0915 13:13:44.740468 I model.py:73] "Calling classifier..."
I0915 13:13:44.757952 I model.py:93] "Classifier done."
I0915 13:13:44.758023 I model.py:104] "Predicted class: 1"
I0915 13:13:44.758040 I model.py:108] "Detected question, calling generator..."
[transformers] Both 'max_new_tokens' (=150) and 'max_length' (=2048) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the
ation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
I0915 13:13:48.884266 I model.py:125] "Generator done."
```

